USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Escola Técnica Superior de Enxeñaría (ETSE)

Sistemas Operativos II

Grado en Ingeniería Informática

Informe Técnico: Apartado 1

Carreras Críticas en el Problema del Productor-Consumidor

Autores: [Alan Barreiro Martínez - Adriana Sánchez-Bravo Cuesta]
Asignatura: Sistemas Operativos II
Fecha: 20 de abril de 2026

Índice

1. Introducción **2**

2. Funcionamiento del código **2**

3. Ejecución **3**

4. Análisis de resultados **3**

5. Conclusiones **5**

1. Introducción

En este apartado se estudiarán las consecuencias de las carreras críticas en un programa que resuelve el problema del productor-consumidor con números enteros.

El código funciona a través de dos tipos de hilos, los hilos productores y los consumidores. Dependiendo del tipo de hilo, realizará distintas funciones. Las funciones de los productores se basan en leer los números del archivo de texto y añadirlos al buffer compartido por los hilos, mientras que los consumidores retiran esos números de la memoria compartida y los intercambian por un '0'. Además, tanto los productores como los consumidores deben sumar los elementos que van leyendo, y al final del código se deberá imprimir la suma de cada uno.

En este informe se compararán los resultados entre el código utilizando semáforos para evitar carreras críticas, y el código sin el semáforo que controla el acceso al buffer compartido.

2. Funcionamiento del código

Para ambos códigos las funciones `produce_item()` y `remove_item()` son iguales, ya que estas no necesitan semáforos. En ellas principalmente se leen los enteros del archivo y del buffer compartido.

Las principales diferencias se encuentran en las funciones insert-item() y consume-item().

Dentro de la función insert-item(), la primera diferencia se encuentra al principio del bucle principal, ya que al quitar el semáforo que regula la entrada al buffer compartido se desprotege la entrada y muy probablemente produzca carreras críticas.

```
while (1){
    sem_wait(&huecos);
    //sem_wait(&sem_buffer);
}
```

Además, en esa misma función, para potenciar la probabilidad de carrera crítica, se pueden añadir sleeps, haciendo que el productor sea más lento que el consumidor. Con esto, y al no tener protegida la entrada a memoria compartida, el consumidor es más rápido que el productor, haciendo que el productor cambie el contador del número de elementos del buffer, y provocando que el consumidor acabe consumiendo elementos que ya había tratado previamente.

```
//se mete el numero leido en el buffer y se aumentan los contadores
sleep(1);
buffer[final] = num;
sleep(1);
final= (final+1) %N;
num_elementos++;
producidos++;
```

Esto se puede observar muy bien ya que cuando el consumidor retira un número del buffer, lo cambia por un 0, por lo que cuando se producen carreras críticas los hilos consumidores acabarán consumiendo 0s. Los resultados se mostrarán en el apartado 3 de resultados. Para la función consume-item(), al igual que en la anterior, en el bucle principal simplemente se debe eliminar el semáforo de control de entrada al buffer.

```
while (1) {
    sem_wait(&elementos);
    //sem_wait(&sem buffer);

    int num= remove_item();
    if (num== -1){
        num_elementos--;
        principio = (principio + 1) % N;

        //sem_post(&sem_buffer);
        sem_post(&huecos);
        break;
    }
    printf("\tConsumidor %d saca '%d' (%d)\n", id, num, consumidos);

    // contar
    suma_cons+= num;
```

Esto hace que exista la posibilidad de que varios consumidores ejecuten remove-item() a la vez, lo cual hace que se puedan consumir elementos varias veces, haciendo que se pierdan datos. Los hilos incrementan los contadores a la vez, y se acaban perdiendo posiciones del buffer.

3. Ejecución

Para la ejecución del código simplemente se debe crear el ejecutable con:

```
$ gcc ej1.c -o ej1 -pthread
$ ./ej1
```

"-pthread" se utiliza cuando un programa trabaja con hilos para enlazar la librería, aunque en este caso también funciona sin añadírselo

4. Análisis de resultados

En los resultados se puede observar perfectamente los efectos que las carreras críticas tienen en el funcionamiento de un programa. Estos experimentos se realizaron con 3 hilos productores y 3 hilos consumidores, además de un archivo de texto que cuenta con 4 números enteros.

Estos son los resultados cuando el código cuenta con los semáforos necesarios para un comportamiento normal:

```
Numero de productores: 3
Numero de consumidores: 3
Productor 1 mete '1' (1)
Productor 2 mete '2' (2)
        Consumidor 1 saca '1' (1)
Productor 3 mete '33' (3)
Productor 1 mete '4' (4)
        Consumidor 2 saca '2' (2)
        Consumidor 1 saca '33' (3)
        Consumidor 3 saca '4' (4)

Total producidos: 4
Suma productor= 40
Total consumidos: 4
Suma del consumidor= 40
```

Se puede ver que el funcionamiento es correcto, los productores van leyendo el archivo y añadiendo los enteros al buffer compartido, mientras que los consumidores los retiran. Aunque el orden no sea correcto, esto no afecta en el resultado.

Los siguientes resultados son obtenidos quitando los semáforos “sem-buffer”:

```
Numero de productores: 3
Numero de consumidores: 3
Productor 3 mete '33' (1)
Productor 2 mete '2' (1)
Productor 1 mete '1' (1)
        Consumidor 1 saca '2' (1)
        Consumidor 2 saca '2' (2)
        Consumidor 3 saca '2' (3)
Productor 2 mete '4' (2)
        Consumidor 2 saca '0' (4)
        Consumidor 1 saca '0' (4)
        Consumidor 3 saca '0' (5)
        Consumidor 2 saca '0' (6)
^C
```

Además de que el código se queda en un bloqueo indefinido debido a la desincronización de los hilos, también se puede ver la condición de carrera en el acceso al buffer.

Varios consumidores acceden a la vez a la posición donde se encuentra el entero 2, lo que provoca la retirada repetida del elemento. Asimismo, al no existir exclusión mutua, el contenido del buffer, más concretamente, las variables "principio" y "num-elementos" se desincronizan con el orden real que debería seguir el buffer.

Esto provoca que los semáforos pierdan coherencia con cómo debería estar el buffer, y dejando a los hilos bloqueados.

5. Conclusiones

En conclusión, teniendo en cuenta los resultados, se puede confirmar que la eliminación del semáforo que protege el acceso al buffer provoca condiciones de carrera crítica, generando accesos no controlados a variables compartidas.

Esto produce inconsistencias en el buffer, como consumos duplicados, valores incorrectos y desincronización de las variables contadoras. En consecuencia, el programa produce resultados erróneos e incluso se producen bloqueos permanentes , demostrando la importancia del control de la memoria compartida.